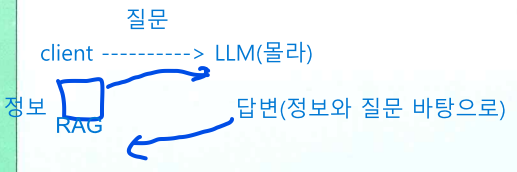
function calling: 최신 정보 llm이 주입
RAG: llm이 모르는 정보를 내부에서 제공.

09장

RAG로 문서에 기반해 답변하는 챗봇 만들기

학습하지 못한 정보 ----> LLM

GPT와 같은 언어 모델은 주어진 텍스트를 바탕으로 확률에 따라 다음에 나올 문장을 적절하게 예측합니다. 그래서 LLM이 학습되지 않은 ~~최신~~ 정보에 대해선 정확한 답변을 하지 못합니다. 사실이 아닌 정보를 그럴듯하게 말해서 혼란을 일으키기도 하죠. 이번 장에서는 RAG를 활용해서 이러한 언어 모델의 한계를 극복하는 방법을 배워 보겠습니다. 환각



R: 검색
A: 증강
G: 생성

09-1 RAG란 무엇일까?
09-2 RAG 기반한 챗봇 구현하기
09-3 스트림릿으로 챗봇 완성하기

09-1 RAG란 무엇일까?

RAG가 무엇인지 알아보고 RAG를 활용하기 위해 필요한 개념을 살펴보겠습니다.

LLM
이전문장 -> [ ] -> 다음 단어

언어 모델과 RAG의 작동 방식

context(문맥)
Attention -> softmax

GPT와 같은 언어 모델은 이전 문장들을 바탕으로 다음에 나올 문장을 확률적으로 계산하여 가장 가능성이 높다고 판단되는 단어와 문장을 생성합니다. 언어 모델은 기억하거나 생각을 LLM
하는 것이 아니라 과거에 학습한 데이터를 기반으로 계산된 결과를 내놓을 뿐이죠. 그래서 엉뚱한 답변을 하는 환각 현상이 발생합니다.

어쩌면 우리는 GPT와 같은 언어 모델에게 사람보다 더 가혹한 요구를 하고 있을지도 모릅니다. 사람도 기억이 가물가물하거나 확신이 없는 상태에서는 종종 잘못된 답변을 하기도 하니까요. 예를 들어 국정 감사를 받는 장관을 생각해 봅시다. LLM 장관은 실수를 줄이기 위해 RAG 보좌진을 배석시킵니다. 질문을 받으면 보좌진은 필요한 정보를 빠르게 검색해서 장관에게 전달하고, 장관은 구체적인 정보를 근거로 대답을 합니다. R
RAG 근거로 LLM이 답변

RAG도 이와 비슷한 방식으로 작동합니다. 리트리버가 데이터 조각들을 미리 잔뜩 쌓아 두고 R
질문이 들어오면 유사한 데이터 조각을 신속하게 찾아 언어 모델에게 건네며 이것을 기반으로 답변하라고 알려 주는 방식이죠. 이렇게 하면 언어 모델이 학습하지 않은 정보에 대해서도 답변할 수 있고 실수할 확률이 크게 줄어듭니다.

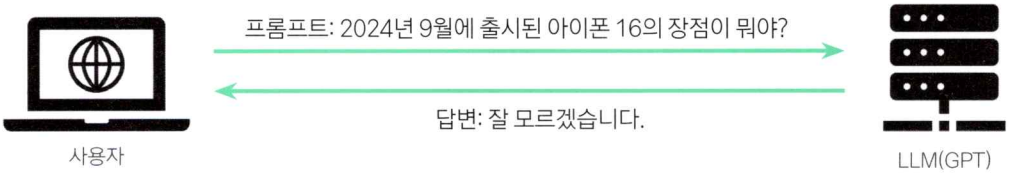
chunk: 단위
문장에 대한 벡터



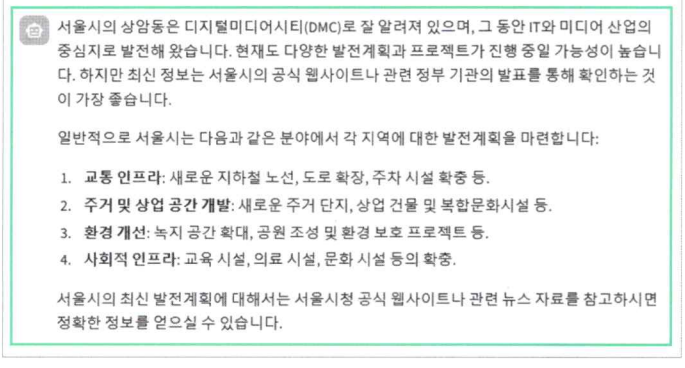
리트리버의 도움을 받아 답변하는 언어 모델

기본적인 언어 모델의 답변과 RAG의 차이

GPT 같은 언어 모델에 질문하면 언어 모델은 그 질문의 답이 될 가능성이 높은 내용을 기반으로 답변을 구성합니다. 만약 2024년 5월에 학습을 완료한 모델이라면 그 이후에 나온 정보는 제대로 답변할 수 없겠죠. 잘 모른다고 대답하거나 일반적인 내용을 답변할 가능성이 높습니다.



이런 경우 답변을 하더라도 불안하기는 마찬가지입니다. 예를 들어 GPT에게 '서울시의 상암동 발전 계획이 뭘까?'라고 질문했더니 다음처럼 그럴듯하게 답변했습니다. 하지만 이 답변은 상암동의 구체적인 발전 방향이라기보다 일반적인 도시 발전 계획으로 보입니다.


서울시의 상암동은 디지털미디어시티(DMC)로 잘 알려져 있으며, 그 동안 IT와 미디어 산업의 중심지로 발전해 왔습니다. 현재도 다양한 발전계획과 프로젝트가 진행 중일 가능성이 높습니다. 하지만 최신 정보는 서울시의 공식 웹사이트나 관련 정부 기관의 발표를 통해 확인하는 것이 가장 좋습니다.

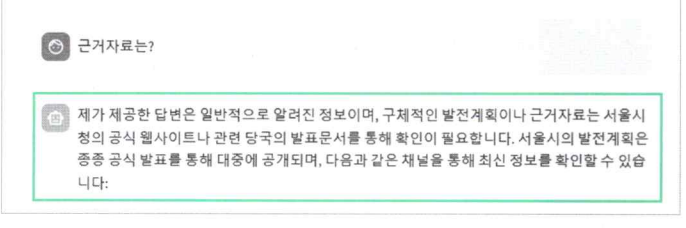
일반적으로 서울시는 다음과 같은 분야에서 각 지역에 대한 발전계획을 마련합니다:

1. **교통 인프라**: 새로운 지하철 노선, 도로 확장, 주차 시설 확충 등.
2. **주거 및 상업 공간 개발**: 새로운 주거 단지, 상업 건물 및 복합문화시설 등.
3. **환경 개선**: 녹지 공간 확대, 공원 조성 및 환경 보호 프로젝트 등.
4. **사회적 인프라**: 교육 시설, 의료 시설, 문화 시설 등의 확충.

서울시의 최신 발전계획에 대해서는 서울시청 공식 웹사이트나 관련 뉴스 자료를 참고하시면 정확한 정보를 얻으실 수 있습니다.

GPT가 질문에 맞는 구체적인 답변을 생성하지 못하고 일반적인 답변을 생성한 경우

그것마저도 질문 내용과 관련 있는 정보에 기반한 답변인지 확인하기 어렵습니다.


근거자료는?

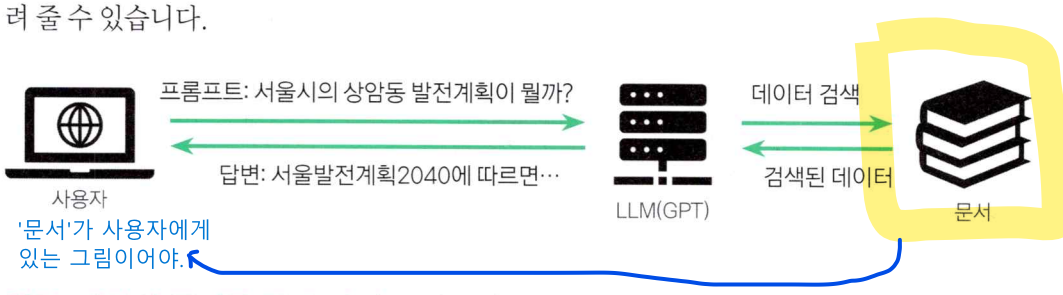
제가 제공한 답변은 일반적으로 알려진 정보이며, 구체적인 발전계획이나 근거자료는 서울시청의 공식 웹사이트나 관련 당국의 발표문서를 통해 확인이 필요합니다. 서울시의 발전계획은 종종 공식 발표를 통해 대중에 공개되며, 다음과 같은 채널을 통해 최신 정보를 확인할 수 있습니다:

근거 있는 정보인지 확인하기 어려운 경우

기존 RAG는 질문해서 찾는 성능이 떨어짐.
RAG 성능 개선 어떻게 했는지.

50% 정도만 해결됨.

RAG를 사용하면 이런 문제를 해결할 수 있습니다. 질문과 관련된 문서를 찾아 그 문서를 바탕으로 답변을 제공하니까요. 그리고 답변을 만드는 데 사용한 문서의 출처도 사용자에게 알려 줄 수 있습니다.



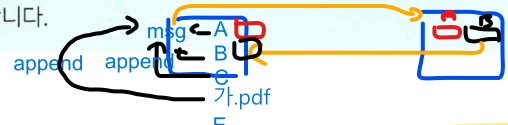
청킹: 대량의 문서를 ~~쪽~~ 단위로 자르기

특정 크기

GPT에게 많은 양의 문서를 한 번에 텍스트로 전달해 답을 하라고 요청할 수도 있지만 이는 좋은 방법이 아닙니다. 여러 가지 이유가 있지만 주로 다음 세 가지 문제가 있습니다.

context window

1. 언어 모델이 한 번에 처리할 수 있는 텍스트 길이를 초과할 수 있습니다.
2. 언어 모델이 문서에서 필요한 정보를 제대로 찾지 못할 수 있습니다.
3. 대화형 방식으로 진행할 경우 매번 수백 페이지의 문서를 언어 모델에 입력하게 되므로 높은 토큰 비용이 발생합니다.



pdf가 append 되어 보내져 토큰이 많이 사용됨.
-> pdf 잘라(청킹) 필요한 내용만 보내기.

컨텍스트 윈도우context window는 언어 모델이 한 번에 처리할 수 있는 텍스트 길이의 한계를 뜻합니다. 이 길이를 초과하는 텍스트를 입력하면 언어 모델이 처리하지 못해 오류가 나거나 제공한 텍스트의 뒷부분만을 사용해 답을 하게 됩니다. GPT는 컨텍스트 윈도우 길이가 수십에서 수백 쪽으로 길어졌지만 여전히 한계는 존재하며 다른 언어 모델도 마찬가지입니다.

문서에 기반해서 대화하려고 언어 모델에 수백 쪽에 달하는 문서를 넣으면 언어 모델이 데이터에서 필요한 부분을 찾아내기 어렵습니다. 이런 문제는 사람도 마찬가지입니다. 예를 들어 국정 감사를 받는 장관에게 300쪽짜리 보고서를 넘겨주고 '이 내용으로 답하세요'라고 하면 제대로 확인하고 답변하기 어렵겠죠. 언어 모델도 마찬가지로 데이터가 너무 많으면 필요한 정보를 찾는 데 어려움이 있습니다.

또한 PDF 한 쪽당 약 800토큰이 소모된다고 할 때 300쪽이라면 약 24만 토큰에 해당합니다. 2025년 4월 기준으로 GPT-4o는 100만 토큰당 비용이 5달러이므로 언어 모델에 PDF 문서를 한번 입력하는 데 약 1천 원 소모됩니다. 만약 대화형으로 이 작업을 반복한다면 비용이 매번 1천 원씩 발생하므로 금방 많은 비용이 소진될 수 있습니다.

RAG는 이런 문제를 해결하기 위해 대량의 문서를 ~~쪽지~~ 특정 단위로 나누는 청킹chunking 작업을 합니다. 이렇게 나누어진 문서 조각을 청크chunk라고 합니다.



대량의 문서를 쪽지 단위로 나누는 청킹

긴 내용은 청크로 잘려서 일부분만 나갈 수 있음.

context window에 따라 청크의 크기 달라짐.

청크의 크기는 문서나 언어 모델에 따라 다르며, 일반적으로 언어 모델의 토큰당 비용과 처리할 수 있는 길이를 고려하여 결정합니다. 특이한 경우도 있습니다. 예를 들어 동요 수백 곡의 가사가 적혀 있는 문서는 곡마다 각각 청크로 나누는 것이 더 적합할 수 있습니다. 다른 곡과 섞여 한 청크에 포함되면 언어 모델이 답을 할 때 오히려 방해가 될 수도 있으니까요.

짧은 내용은 같이 청크로 묶여 혼동될 수 있음.

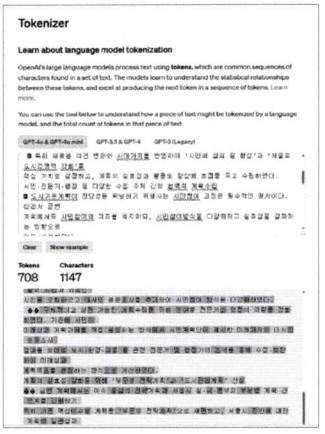
RAG는 이렇게 청크 단위로 문서를 나누고 사용자의 질문과 가장 유사한 청크 몇 개를 찾아 질문과 함께 언어 모델에 전달하여 문서에 기반한 답변을 생성합니다.

한 걸음 더! 토큰은 어떻게 계산하나요?

토큰화tokenizing는 문장이나 문단 같은 긴 텍스트를 기계가 이해할 수 있는 작은 단위인 토큰token으로 분해하는 과정을 말합니다. 한국어 문장을 단어, 어절, 서브워드subword 단위로 나누는 과정이 토큰화입니다. 예를 들어 '나는 커피를 마신다.'라는 문장을 '나', '는', '커피', '를', '마신다'처럼 나눌 수도 있고 자모 단위까지 쪼갤 수도 있습니다. 상황과 목적에 따라 다양한 수준으로 토큰화가 이뤄집니다.

토큰화는 반드시 사람이 이해할 수 있는 형태로 이루어지는 것은 아닙니다. 자연어 처리NLP 모델에 따라 텍스트를 쪼개는 규칙이 다릅니다. 오픈AI에서 토큰화하는 방법으로 계산하려면 오픈AI의 토크나이저 페이지(https://platform.openai.com/tokenizer)를 이용하면 됩니다.

또한 텍스트를 토큰화하는 것과 문서를 청킹하는 것은 다른 개념입니다. 문서 청킹은 수십, 수백 쪽의 텍스트를 일정한 양으로 조각내는 것을 의미하고, 이때 청크를 나누는 기준으로 일정한 토큰 단위 혹은 글자 수를 사용합니다.



오픈AI에서 제공하는 토큰 계산기

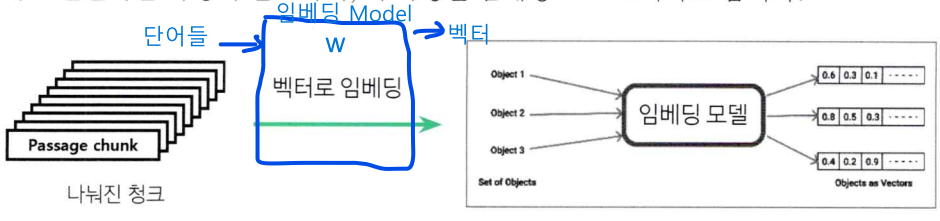
임베딩: 숫자 -> 계산 가능
-> 유사한 텍스트 찾기 가능

텍스트 -> 숫자(숫자 1개.): 인코딩
텍스트(단어) -> 벡터(리스트. 숫자 여러 개.): 임베딩

임베딩: 텍스트를 벡터로 변환하기

잘려진 문서.

문서를 청크 단위로 나눈 후 사용자 질문과 관련 있을 가능성이 높은 청크를 어떻게 찾을 수 있을까요? 예를 들어 '서울시의 상암동 발전 계획이 뭘까?'와 같은 질문에 답변하려면 문서에서 수백, 수천 개나 되는 청크 중 가장 관련 있는 몇 개를 선택해야 합니다. 청크는 보통 5개 정도를 선택하는데 이때 개수는 여러 번 실험을 해서 결정합니다. 그 후 텍스트 간의 유사도를 비교하는 과정을 거칩니다. 유사도를 수학적으로 비교하려면 텍스트를 상호 비교할 수 있도록 같은 단위의 수치 정보로 바꿔야 하죠. 이를 위해 각 청크의 텍스트를 벡터 공간에 숫자 형태로 변환하는 과정이 필요하며, 이 과정을 임베딩embedding이라고 합니다.



텍스트를 벡터로 변환하는 임베딩

특정 분야별 특화된 임베딩 모델이 있음.
만약 금융회사면 금융회사 특화 임베딩 모델 쓰는 게 좋음.

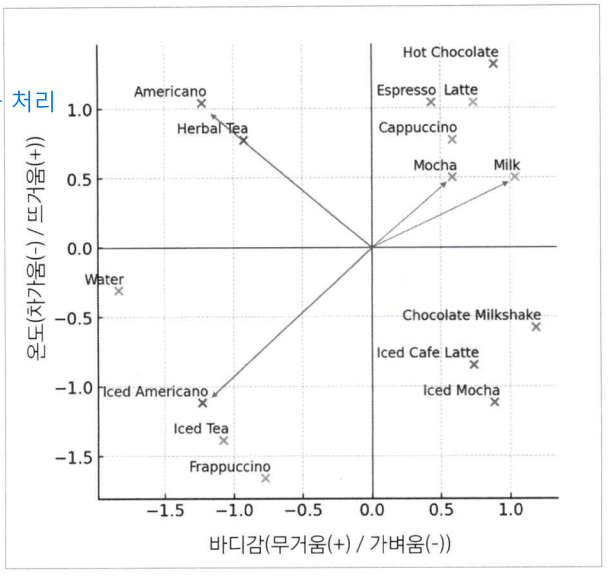
이 과정에서 새로운 2가지 용어 '벡터'와 '임베딩'이 나왔습니다. 이 용어를 이해하기 위해 잠시 수학 이야기를 해보겠습니다.

벡터와 임베딩

벡터는 어려운 개념이 아닙니다. 정보를 수치로 표현하여 일렬로 나열한 것을 벡터라고 할 수 있습니다. 예를 들어 [키, 몸무게, 위도, 경도, 출생연도]라는 원소가 5개인 벡터로 사람을 표현하면 마이클 조던은 [198, 98, 40.67, -73.94, 1963], 일론 머스크는 [188, 82, 30.27, -97.74, 1971] 로 표현할 수 있죠. 물론 이 숫자 5개만으로 그 사람을 완전히 설명할 수는 없지만 비교하기 쉽게 5개 요소를 사용한 것입니다. 더 수학적인 용어로는 5차원 벡터 공간에 정보를 표현한 것이죠. 다시 말해 사람 정보를 5차원 벡터로 임베딩한 것입니다.

이제 이렇게 표현된 벡터들 간의 유사성을 계산하는 방법을 알아보겠습니다. 5차원 벡터 공간을 종이에 그리기는 어려우니 그 대신 2차원 벡터 공간에 단어를 표현하는 방법을 살펴보겠습니다. 예를 들어 카페 음료 메뉴만 있는 세상을 상상해 봅시다. x축은 음료의 바디감(농도), y축은 음료의 온도를 의미합니다. 상대적인 수치로 이 값을 정해서 음료의 위치를 정했습니다. 이렇게 음료 종류를 2차원 벡터 공간으로 임베딩하여 표현할 수 있습니다.

정규화 처리

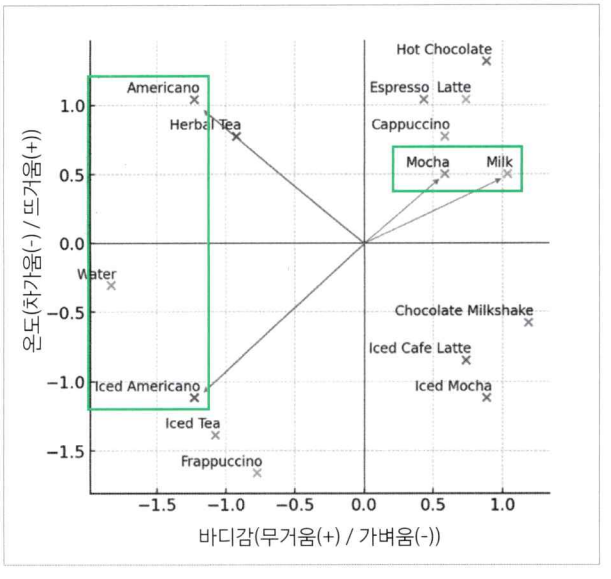


바디감과 온도를 기준으로 카페 음료 메뉴를 2차원 벡터 공간에 표현한 예

코사인 유사도

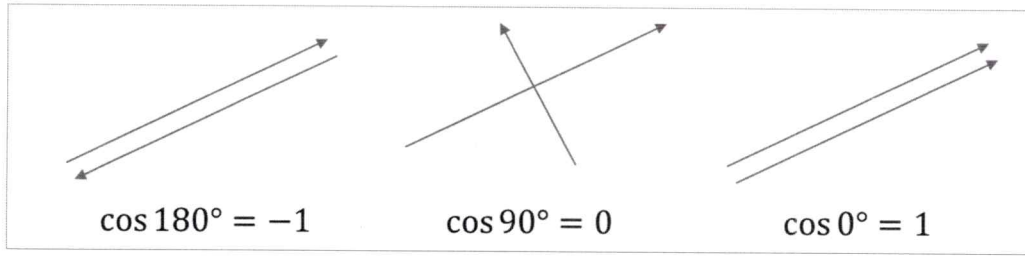
다음 그래프에서 모카Mocha와 우유Milk를 나타내는 두 벡터가 이루는 각도는 아메리카노Americano와 아이스 아메리카노Iced Americano를 나타내는 두 벡터 사이의 각도보다 훨씬 좁습니다. 이것은 모카와 우유의 관계가 아메리카노와 아이스 아메리카노의 관계보다 더 유사하다는 것을 의미합니다.

✦ 지금 당장 이 결과에 동의하기 어려울 수도 있지만, 이 내용을 모두 배우고 나면 이런 예시를 제시한 이유를 알 수 있을 것입니다.



음료 간의 유사도 비교

이렇게 두 벡터 사이의 각도를 이용하면 음료 간의 유사도를 계산할 수 있습니다. 각도가 180°에 가까우면 유사도가 낮고 각도가 0°에 가까우면 유사도가 높다는 의미입니다. 유사도는 -1에서 1 사이의 값으로 표현하며, 1에 가까울수록 두 벡터가 더 유사하다고 볼 수 있습니다.



유사도를 계산하는 방법을 코사인 유사도cosine similarity라고 합니다. 다음 수식을 고등학교 기하학 시간에 배웠을 텐데요. 벡터 A와 벡터 B가 있을 때 두 벡터의 내적을 각 벡터의 길이를 곱한 값으로 나누면 두 벡터의 코사인 값을 구할 수 있습니다. 다음 수식에서 |A|는 벡터 A의 길이, |B|는 벡터 B의 길이, A · B는 벡터 A와 벡터 B의 내적을 의미합니다.

✦ 내적은 두 벡터의 대응되는 원소들을 곱한 후 그 결과를 모두 더한 값을 의미합니다.

$$\cos(\theta) = \frac{A \cdot B}{|A||B|}$$

모카와 우유의 코사인 유사도를 계산해 보겠습니다. 모카와 우유의 x값과 y값이 다음과 같다고 합시다.

$$V_{mocha} = [0.584 \;\; 0.504]$$
$$V_{milk} = [1.037 \;\; 0.504]$$

벡터의 길이를 의미하는 $|V_{mocha}|$와 $|V_{milk}|$를 구해 보면 다음과 같습니다.

$$V_{mocha} = \sqrt{0.584^2 + 0.504^2} = 0.771$$
$$V_{milk} = \sqrt{1.037^2 + 0.504^2} = 1.153$$

이제 두 벡터의 내적을 구해 보면 다음과 같습니다.

$$V_{mocha} \cdot V_{milk} = 0.584 \cdot 1.037 + 0.504 \cdot 0.504 = 0.856$$

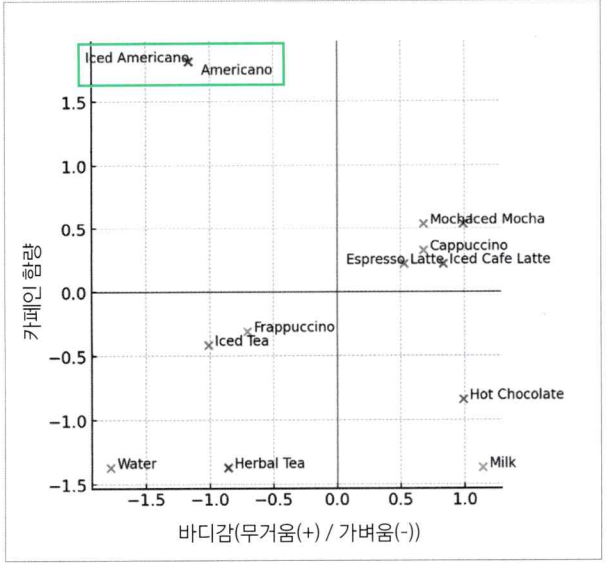
이제 코사인 유사도를 계산해 봅시다.

$$\cos(\theta) = \frac{A \cdot B}{|A||B|} = \frac{0.856}{0.771 \cdot 1.153} = 0.966$$

물론 RAG를 사용할 때 이 과정을 직접 계산할 필요는 없습니다. 이미 다른 사람들이 만들어 놓은 도구를 사용하면 되니까요. 중요한 것은 텍스트의 유사도를 계산하는 개념을 이해하는 것입니다. 종이에서는 2차원만 표현할 수 있지만 3차원, 5차원, 심지어 1024차원까지 확장되더라도 이 개념은 그대로입니다. 벡터의 유사도를 계산하는 방식은 코사인 유사도뿐만 아니라 유클리디안 유사도Euclidean similarity 등 다양하지만 벡터로 변환된 수치를 이용해 유사도를 계산한다는 개념은 마찬가지입니다.

임베딩 모델의 중요성 임베딩 모델 만들려 하지 말고 있는 거 쓰기..

임베딩을 통해 벡터로 변환된 데이터를 기반으로 유사도를 계산할 수 있지만 임베딩 방식에 따라 유사도의 결과는 달라질 수 있습니다. 예를 들어 카페 음료 메뉴를 바디감과 온도를 기준으로 임베딩했을 때 그 결과가 만족스럽지 않을 수 있습니다. 앞에서처럼 아이스 아메리카노와 아메리카노보다 카페 모카와 우유가 더 유사한 음료라는 결과가 나올 수 있죠. 만약 바디감과 온도가 아니라 다음 그림처럼 바디감과 카페인 함량으로 표현하면 아이스 아메리카노와 아메리카노는 2차원 벡터 공간에서 거의 동일한 위치에 표현됩니다. 이처럼 벡터 공간에서 유사도는 임베딩을 어떻게 하는지에 따라 완전히 다르게 평가될 수도 있습니다.



바디감과 카페인 함량을 기준으로 카페 음료 메뉴를 2차원 벡터 공간에 표현한 예

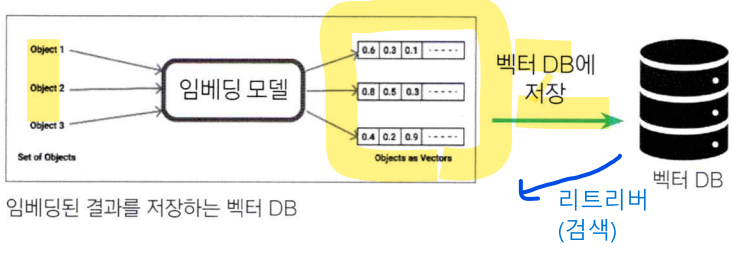
물론 위의 예제는 텍스트를 개념적으로 2차원 평면에 표현했을 뿐입니다. 실제로 텍스트를 임베딩하는 모델은 훨씬 더 고차원의 벡터 공간으로, 인간이 직관적으로 이해하기 어려운 방식으로 데이터를 변환합니다. 임베딩 방법은 매우 다양하므로 텍스트를 벡터로 변환하는 모델도 수없이 존재합니다. 또한 언어나 특정 분야에 따라 더 적합한 임베딩 모델이 있을 수 있으며, 필요한 경우 모델을 파인튜닝하여 성능을 개선할 수도 있습니다.

검색, 유사도 찾아 꺼내주기도 함.
(내가 유사도 계산할 필요x)

벡터DB { 저장: 청크 -> 벡터화 -> 저장
검색: 질문(text) -> 벡터화 -> 검색(질문 넣어 검색)

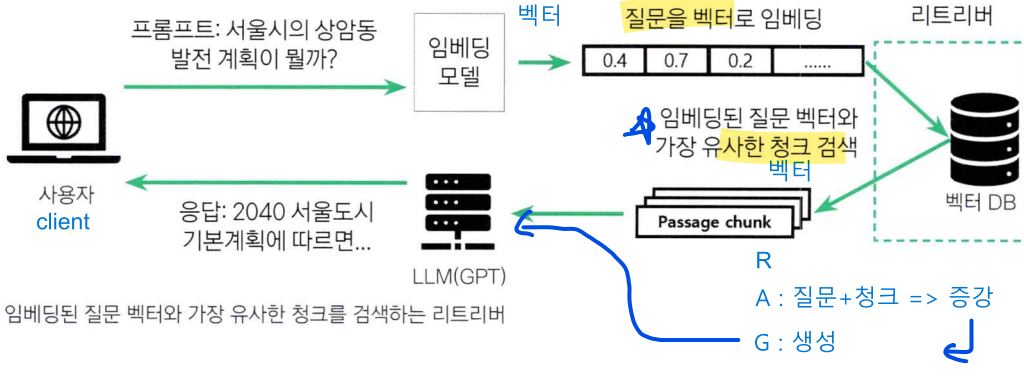
벡터 DB와 리트리버

벡터 DB[Vector DB]는 벡터로 변환된 결과를 저장하고 텍스트 간의 유사도를 계산하여 사용자가 입력한 질문과 가장 유사한 청크를 찾아낼 때 유용합니다. 벡터 DB는 고차원 벡터에서 의미의 유사성을 빠르게 계산하고 그 결과를 반환할 수 있도록 최적화되어 있으며, 크로마 DB[Chroma DB], FAISS[Facebook AI Similarity Search], 파인콘[Pinecone] 등이 있습니다. 이 책에서는 크로마 DB를 사용해 RAG를 구성할 계획입니다.

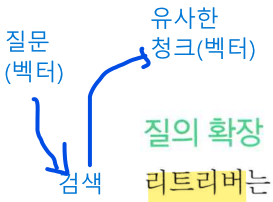


임베딩된 결과를 저장하는 벡터 DB

리트리버[retriever]는 사용자가 질문한 정보에 적절한 답을 생성하는 데 필요한 데이터를 가져오는 역할을 합니다. 벡터 DB에 저장된 내용 중에서 유사도가 가장 높은 청크를 찾아서 가져옵니다.



임베딩된 질문 벡터와 가장 유사한 청크를 검색하는 리트리버



질의 확장

리트리버는 사용자의 질의를 벡터로 변환한 뒤 그 벡터와 유사성이 높은 청크를 검색합니다. 그래서 사용자의 질의가 명확하지 않고 이해하기 어려우면 적절한 청크를 가져올 확률이 낮아집니다. 예를 들어 RAG를 사용한 대화가 다음처럼 이어진 상황을 가정해 봅시다.

> 사용자: 벡터 DB가 뭐야?
> AI: 벡터 데이터베이스(Vector Database)는 고차원 벡터 데이터를 저장, 관리 및 인덱싱하는 특수한 데이터베이스입니다.
> 사용자: 쓸 만한 거 추천해 줘.

사용자의 '쓸 만한 거 추천해 줘.'라는 요청은 '벡터 DB 중에서 쓸 만한 것을 추천해 달라.'는 의미입니다. 만약 리트리버가 '쓸 만한 거 추천해 줘.'라는 문장을 그대로 임베딩하여 변환된 벡터로 유사도 검색을 한다면 벡터 DB와 전혀 관련 없는 다른 청크들이 검색될 확률이 매우 높습니다. 따라서 사용자의 마지막 질문을 그대로 검색하는 대신 질문의 문맥을 파악하여 적절한 질문으로 변환한 뒤, 그 질문을 벡터로 임베딩하여 검색하는 과정이 필요합니다. 이렇게 사용자의 질문을 더 명확하게 수정하는 작업을 질의 확장query augmentation이라고 합니다.

09-2 RAG에 기반한 챗봇 구현하기

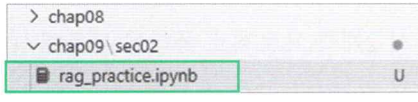


RAG의 기본 개념은 익혔으니 이제 개발을 시작해 봅시다. PDF 문서나 웹 사이트를 기반으로 답변하는 챗봇을 만들 계획입니다. 우선 주피터 노트북에서 단계별로 실습을 진행하고 이후에 스트림릿을 이용해 챗봇 형태로 발전시켜 보겠습니다.

Do it! 실습 PDF 파일 텍스트로 변환하고 청크 단위로 쪼개기

결과 파일: chap09/sec02/rag_practice.ipynb

1. rag_practice.ipynb라는 이름으로 주피터 노트북을 생성합니다.



2. 필요한 랭체인 관련 라이브러리들은 앞선 실습에서 대부분 설치했습니다. 이번 실습에 필요한 pypdf와 langchain_community를 추가로 설치하겠습니다. 첫 셀에 다음과 같이 입력해 라이브러리를 설치합니다.

✦ 만약 이전 실습에서 Langchain 라이브러리를 설치하지 않았다면 설치한 후 진행하세요.

pdf -> 읽기 -> text

라이브러리 임포트하기 rag_practice.ipynb (1)

사용 → PyPDFReader class (읽기만 가능)

```
%pip install pymupdf pypdf langchain_community -> langchain-unstructured 로 바뀔 듯.
```

"나중에 없어진다" 예약

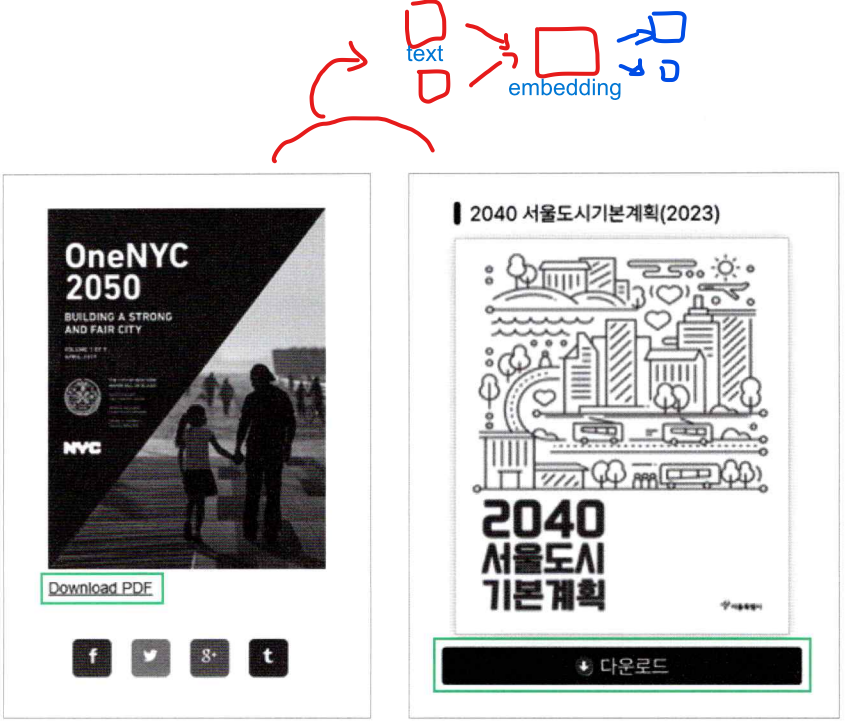
3. RAG를 구현하려면 사용할 문서가 필요합니다. 다음 웹 페이지에서 예시로 사용할 PDF 문서를 각각 내려받습니다.

- 「OneNYC 2050 Building a Strong and Fair City」: https://a860-gpp.nyc.gov/concern/parent/gx41mm584/file_sets/1z40kw69m
- 「2040 서울도시기본계획」: https://urban.seoul.go.kr/view/html/PMNU5020400001?booksType=BK0300

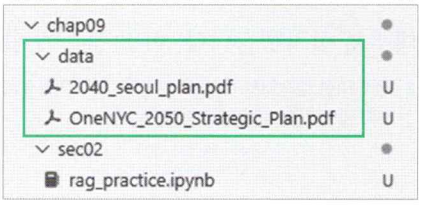
순서

1. pip install pypdf -> 버전문제 발생 가능 (단독사용가능 → pdf 파일 처리 엔진(기능)) pypdf 단독 사용(Langchain과 상관없이 pdf 처리 앱에 사용)
2. pip install langchain-chroma langchain-classic
3. pip install langchain-community

pypdf를 먼저 설치하면 현재 환경에 맞는 버전을 설치.
langchain-community 먼저 설치하면 이 패키지에 필요한 pypdf 설치할 때 최신 버전으로 설치..



4. 프로젝트 폴더 안에 data 폴더를 만들고 내려받은 문서 2개를 프로젝트 폴더에 복사해 넣습니다.

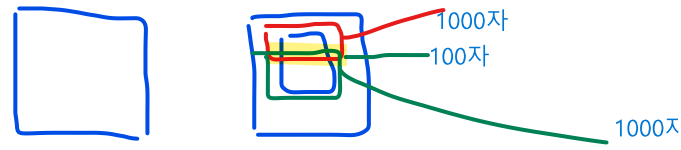


5. 「OneNYC 2050」 문서의 내용을 data_nyc에 담아 텍스트를 추출해 보겠습니다. PyPDFLoader는 랭체인에서 제공하는 모듈로, PDF 파일을 읽어 텍스트 데이터를 추출하고 PDF 내용을 랭체인의 다른 도구들과 연결하여 사용할 수 있습니다.

pyPDFLoader 사용하기 — rag_practice.ipynb (2)

```
from langchain_community.document_loaders import PyPDFLoader

# PDF 파일을 읽어서 텍스트 데이터 추출
loader = PyPDFLoader('C:/github/gpt_agent_2025_easyspub/chap09/data/OneNYC_2050_Strategic_
Plan.pdf')
data_nyc = loader.load()
print(data_nyc)
```



이 셀을 실행한 결과는 다음과 같습니다.

✦ 페이지 양에 따라 시간이 몇 분 정도 소요될 수 있습니다.

```
[Document(metadata={'source': 'C:/github/gpt_agent_2025_easyspub/chap10/data/OneNYC_
2050_Strategic_Plan.pdf', 'page': 0}, page_content='OneNYC \n2050\nBUILDING A STRONG
\nAND FAIR CITY \nVOLUME 1 OF 9
(... 생략 ...)
```

6. RecursiveCharacterTextSplitter를 사용하여 출력된 긴 텍스트 데이터를 청크 단위로 나눕니다. 이때 텍스트 데이터를 1000자씩 나누고 100자의 오버랩을 설정하여 중요한 정보가 잘려서 사라지지 않도록 합니다. 오버랩은 인접한 청크 간에 중복된 내용을 포함시켜 중요한 정보가 빠지지 않도록 합니다. 예를 들어 텍스트를 1000자씩 자를 때 문장이 중간에서 끊기면 중요한 정보가 사라질 수 있습니다. 이때 100자씩 겹치도록 오버랩을 하면 이 문제를 해결할 수 있습니다. 이렇게 설정한 text_splitter로 문서를 쪼개 all_splits에 담습니다.

출력된 텍스트를 청크로 나누기 rag_practice.ipynb (3)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

# 텍스트 데이터를 1000자 단위로 나눔. overlap은 100자로 설정
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
all_splits = text_splitter.split_documents(data_nyc)
```

7. all_splits를 모두 출력하여 문서가 적절하게 나누어졌는지 확인합니다. 다음 코드를 사용해 청크를 출력합니다.

청크 출력하기 rag_practice.ipynb (4)

```
for i, split in enumerate(all_splits):
    print(f"Split {i+1}:-----------------------------------\n")
    print(split)
```

실행된 결과를 보면 PDF의 내용이 오버랩을 포함하여 적절하게 잘 나누어졌습니다. 구체적인 내용은 PDF의 버전이나 pypdf, 랭체인 버전에 따라 조금씩 달라질 수 있습니다.

```
(... 생략 ...)
VISIT A PUBLIC COMPUTER CENTER.
Access broadband by visiting one of the
City's 500+ free public computer centers,
including libraries, public housing' metadata={'source': 'C:/github/gpt_agent_2024_
book/chap10/data/OneNYC_2050_Strategic_Plan.pdf', 'page': 330}

Split 1025:-----------------------------------
page_content='City's 500+ free public computer centers,
including libraries, public housing
facilities, senior centers, and community
centers in the highest need neighborhoods.
(... 생략 ...)

Split 1022:-----------------------------------

page_content='1
4
2
5
3
For more ways you can get involved, visit NYC.GOV/O neNYC.
Share your story of taking action on social media and tag us at #O neNYC.' metadata=
{'source': 'C:/github/gpt_agent_2025_easyspub/data/OneNYC_2050_Strategic_Plan.pdf',
'page': 330}
Split 1023:-----------------------------------

page_content='32  ¦  O neNYC 2050 : MODERN INFRASTRUCTURE
NYC.GOV/O neNYC
THE CITY OF NEW YORK
MAYOR BILL DE BLASIO
DEAN FULEIHAN
FIRST DEPUTY MAYOR
DOMINIC WILLIAMS
CHIEF POLICY ADVISOR
DANIEL A. ZARRILLI
OneNYC DIRECTOR
OneNYC
Learn more about how we are building a
strong and fair city: NYC.GOV/OneNYC
```

```
Join the conversation on social media
and tag us at #OneNYC' metadata={'source': 'C:/github/gpt_agent_2025_easyspub/data/
OneNYC_2050_Strategic_Plan.pdf', 'page': 331}
```

8. `all_splits`에 담긴 요소들이 어떤 데이터 타입인지 출력해 보면 랭체인에서 제공하는 Document 클래스의 인스턴스임을 알 수 있습니다. 랭체인에서는 외부 문서를 Document 객체를 이용해 저장합니다. Document 클래스는 텍스트 데이터와 관련 메타데이터를 저장하고 관리하는 랭체인의 기본 단위입니다. 이 클래스는 page_content 속성에 텍스트 데이터를, metadata 속성에 해당 텍스트와 관련된 메타데이터를 저장하여 문서를 효과적으로 처리하고 활용하는데 도움이 됩니다.앞으로도 종종 사용할 클래스입니다.

all_splits의 데이터 타입 확인하기 — rag_practice.ipynb (5)

```
print(type(all_splits[0]))
```

```
<class 'langchain_core.documents.base.Document'>
```

9. 「2040 서울도시기본계획」 문서도 앞에서 사용한 코드를 응용해 다음과 같이 문서를 청킹하고 출력합니다.

「2040 서울도시기본계획」 문서 청킹하기 — rag_practice.ipynb (6)

```
loader_seoul = PyPDFLoader('C:/github/gpt_agent_2025_easyspub/chap09/data/2040_seoul_
plan.pdf')
data_seoul = loader_seoul.load()
seoul_splits = text_splitter.split_documents(data_seoul)
for i, split in enumerate(seoul_splits):
    print(f"Split {i+1}:-----------------------------------")
    print(split)
```

실행해 보면 이 문서는 중복되는 내용 없이 페이지 단위로 청크가 끊어져 있습니다. 각 청크를 오버랩을 포함하여 빠지는 내용이 없게 하려던 의도와 달리 페이지 단위로 텍스트가 나누어집니다. 이처럼 문장이 페이지 경계에서 끊어지는 문제가 발생한다면 오버랩이 되도록 하는 코드를 추가로 작성해야 할 수도 있습니다.

```
(... 생략 ...)
Split 2:-----------------------------------
page_content='「 2040 서울도시기본계획 」을 발간하며
지난 3년간 코로나19 팬데믹으로 전 세계가 심각한 타격을 받아왔지만, 대한민국의 수도 서
울은 혁신적인 디지털 기술과 뛰어난 시민 의식, 풍부한 자연환경을 토대로 도시의 가능성과 잠재력을
(... 생략 ...)
에 유연하고 신속하게 대응할 수 있는 체계를 통해 현장에서 강력하게 작동하는 수단이 될 것입니다. 미
래지향적인 계획철학을 토대로 융복합적인 토지이용제도를 실현하고 수변녹지와 연계된 생활공간을 조성
하는 등 도시계획체계를 과감하고 유연하게 전환했습니다.' metadata={'source': 'C:/github/gpt_
agent_2024_book/chap10/data/2040_seoul_plan.pdf', 'page': 2}
Split 3:-----------------------------------
page_content='셋째, 미래 변화 속 지속가능한 도시의 구축입니다. 디지털 대전환과 기후위기 등 급변하
는 시
대에 발맞춰 지속가능한 첨단 도시를 구축하기 위한 구체적인 목표를 수립했습니다. 미래 신산업을 육성하고
첨단 미래교통수단의 기반을 마련하는 한편, 자립적인 자원순환체계와 대형재난에 대한 전방위적인 방재체계
를 구축하는 등 미래 변화에 대한 도시의 기술 적응력을 강화하고 안전하고 지속가능한 도시를 만들기 위한
노력을 담아냈습니다.
(... 생략 ...)
```

10. 오버랩이 없이 생성된 청크들에 강제로 오버랩을 처리하겠습니다. 작업을 하기에 앞서 오버랩하지 않은 청크를 살펴보겠습니다. 문서의 앞부분은 차례라서 결과를 제대로 확인하기 어려울 수 있으므로 50번, 51번 청크를 출력합니다. 오버랩한 후에 작업이 잘 되었는지 비교해보세요.

오버랩 처리 전 청크 출력하기 — rag_practice.ipynb (7)

```
print(seoul_splits[50].page_content)
print('----------------------')
print(seoul_splits[51].page_content)
```

```
32제2장 미래상과 목표
9) 대기질 개선과 폐기물 관리 등 광역거버넌스 차원으로 풀어야 할 과제 발생증가하는 생활폐기물 배출량,
다시 우려되는 대기오염Ÿ서울에서 발생하는 폐기물을 처리해 왔던 인천 수도권매립지의 매립 종료가 2025년
예정되면서 폐기물 처리는 주요한 이슈로 떠올랐다
(... 생략 ...)
 병원, 공원 등의 시설도 지속적인 확충 중Ÿ2020년 65세 이상 노년인구는 전체 인구의 15.4%로 최근 10
년 사이 비율이 6.0%p 늘어나면서 노인복지시설에 대한 수요가 증대되고 있다.
----------------------
제1절 서울의 변화진단33-노인여가복지시설 역시 확충되고 있으나, 노령화의 속도를 따라잡지 못해 2020년
에는 노인 천 명당 2.1개소로 2010년에 비해 0.9개소 감소
(... 생략 ...)
```

11. 각 청크는 Document 인스턴스로 생성되어 있고 내용은 page_content에 있습니다. 앞 청크 끝에 뒷 청크의 첫 100자를 추가하기 위해 다음과 같이 코드를 작성합니다.

오버랩 처리 후 청크 출력하기 — rag_practice.ipynb (8)

```
for i in range(len(seoul_splits) - 1):
    seoul_splits[i].page_content += "\n"+ seoul_splits[i + 1].page_content[:100]

print(seoul_splits[50].page_content)
print('----------------------')
print(seoul_splits[51].page_content)
```

12. 다음과 같이 기존의 all_splits에 seoul_splits를 추가하겠습니다.

청크 출력하기 — rag_practice.ipynb (9)

```
print(len(all_splits))
all_splits.extend(seoul_splits)
print(len(all_splits))
```

실행해 보면 「OneNYC 2050」 문서로 만들었던 청크 길이는 1023이었는데, 서울에 관한 문서까지 추가했더니 청크가 1331개로 늘어났습니다. 청크 숫자는 PDF 혹은 랭체인 버전에 따라 조금씩 달라질 수 있습니다.

```
1023
1331
```

한 걸음 더! 청크와 오버랩은 어떻게 설정해야 하나요?

오버랩 길이는 문서의 성격에 맞게 설정해야 합니다. 오버랩이 너무 길면 불필요한 중복이 발생하여 비효율적일 수 있고, 너무 짧으면 언어 모델이 문맥을 제대로 파악하기 어려울 수 있습니다. 다만 개조식으로 작성된 문서라면 오버랩의 길이가 짧더라도 문맥이나 문장이 끊길 가능성이 적습니다. 청크와 오버랩 길이는 문서의 특징에 따라 조정할 수 있습니다. 절대적인 기준은 없으며, 길고 복잡한 문서라면 청크 크기를 1000자에서 2000자 정도로 크게 설정하여 필요한 내용을 충분히 포함하도록 할 수 있습니다. 반면 짧고 간결한 문장으로 작성된 문서는 500자 정도로 작게 설정해도 충분할 수 있습니다.

청크 길이가 길면 컨텍스트 윈도우가 작은 언어 모델에서는 입력 길이에 제한이 있어 처리하기 어려울 수 있습니다. 충분히 똑똑하지 않은 언어 모델에서는 불필요한 정보까지 처리해서 적절한 답변을 생성하지 못할 수도 있죠. 그뿐만 아니라 긴 청크는 더 많은 토큰을 사용하므로 비용이 증가할 수 있습니다. 반대로 청크 길이가 너무 짧으면 답변을 생성하는 데 필요한 정보가 언어 모델에 충분히 전달되지 않을 수 있습니다.

Do it! 실습 오픈AI 임베딩 모델 사용하기

결과 파일: sec02/rag_practice.ipynb

수백 쪽짜리 문서를 청크 단위로 잘랐으니 이제 이를 벡터로 임베딩할 차례입니다. 이 실습에서는 벡터 DB로 크로마 DB를 사용하겠습니다.

1. 크로마 DB를 사용하기 위해 `langchain_chroma`를 셀에 입력하여 설치합니다.

크로마 DB를 사용하기 위한 라이브러리 설치하기(1) rag_practice.ipynb (10)

```
%pip install langchain_chroma
```

만약 다음처럼 오류가 발생한다면 C/C++ 컴파일러를 설치해야 합니다. langchain_chroma는 벡터 계산, 텍스트 처리, 모델과의 상호작용을 위한 수학적 연산에 Numpy 라이브러리를 활용합니다. Numpy를 설치하는 과정에서 C/C++ 컴파일러가 필요합니다.

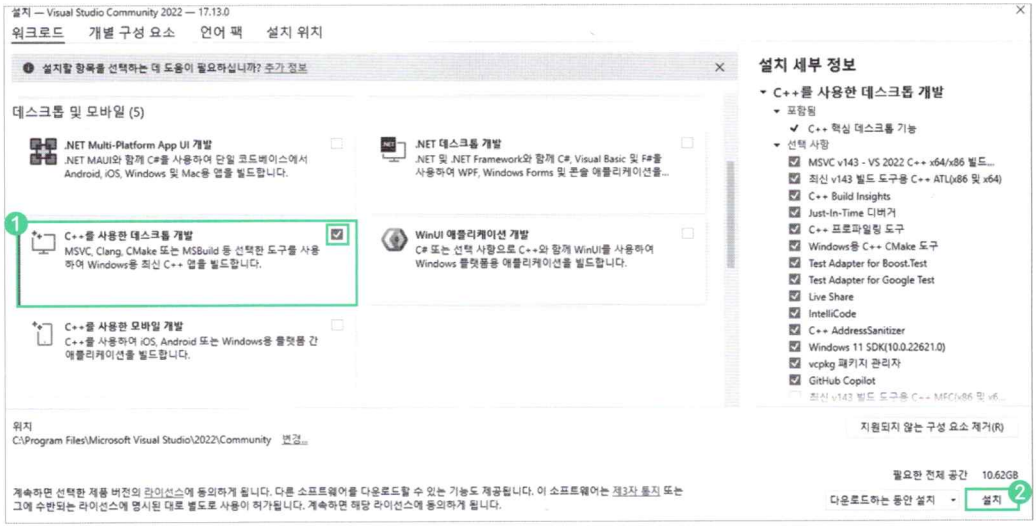
```
error: subprocess-exited-with-error

× Preparing metadata (pyproject.toml) did not run successfully.
| exit code: 1
└─> [21 lines of output]
    + c:\Users\user\Downloads\LLM\LLMtest\venv\Scripts\python.exe C
    The Meson build system
    Version: 1.2.99
    Source dir: C:\Users\user\AppData\Local\Temp\pip-install-4zgcfa
    Build dir: C:\Users\user\AppData\Local\Temp\pip-install-4zgcfac
    Build type: native build
    Project name: NumPy
    Project version: 1.26.4
    WARNING: Failed to activate VS environment: Could not find C:\P
```

2. 오류를 해결하기 위해 Visual C++ Build Tool을 설치하겠습니다. 비주얼 스튜디오 웹 사이트(https://visualstudio.microsoft.com/ko/downloads/)에서 파일을 내려받아 비주얼 스튜디오를 설치합니다.



3. 설치 화면에서 [C++를 사용한 데스크톱 개발]에 체크하고 [설치]를 클릭합니다.

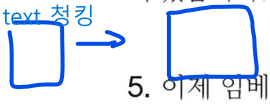


4. 설치가 끝나면 셀을 다시 실행해 라이브러리 설치를 완료합니다.

크로마 DB를 사용하기 위한 라이브러리 설치하기(2) rag_practice.ipynb (10)

```
%pip install langchain_chroma
```

이제 청크들을 벡터로 임베딩하고 이를 벡터 DB에 저장하는 작업을 진행하겠습니다. 텍스트를 벡터로 임베딩하는 모델은 여러 가지가 있습니다. 여기서는 오픈AI에서 API로 제공하는 임베딩 모델인 OpenAIEmbeddings를 사용하겠습니다. 이 임베딩 모델을 사용하려면 API 키가 필요합니다. 임베딩은 오픈AI에서 제공하는 모델뿐만 아니라 BGE-m3 등 다양한 모델이 있습니다. 다만 한 임베딩 모델을 선택하면 계속 같은 임베딩 모델을 사용해야 합니다.



5. 이제 임베딩 모델을 사용해 보겠습니다. `OpenAIEmbeddings`의 인스턴스 `embedding`을 만들고 `embedding.embed_query()`를 사용하여 원하는 텍스트를 벡터로 변환합니다. 이때 임베딩 모델은 `text-embedding-3-large`를 사용하고 오픈AI의 API 키는 .env에서 불러옵니다. 이렇게 설정한 후 `.embed_query()`에 원하는 텍스트를 넣으면 벡터로 변환된 값을 반환합니다.

텍스트를 벡터로 변환하기 — rag_practice.ipynb (11)

```
from langchain_openai import OpenAIEmbeddings

from dotenv import load_dotenv
import os

load_dotenv()

OPENAI_API_KEY = os.getenv('OPENAI_API_KEY')
embedding = OpenAIEmbeddings(model='text-embedding-3-large', api_key=OPENAI_API_KEY)
v = embedding.embed_query("뉴욕의 온실가스 저감 정책은 뭐야?")
print(v)
print(len(v))
```

실행하면 다음과 같이 숫자로 이루어진 벡터가 출력됩니다.

```
[-0.0054374272003769875, -0.04920661821961403,
(... 생략 ...)
0.015299133956432343]
3072
```

한 걸음 더! 오픈AI 임베딩 모델 비교

현재 오픈AI에서 추천하는 임베딩 모델은 text-embedding-3-small, text-embedding-3-large, text-embedding-ada-002입니다. 다음은 2025년 4월을 기준으로 한 임베딩 모델별 성능과 가격입니다

임베딩 모델별 성능 및 가격 비교

모델명	1달러당 페이지 수	MTEB 평가 성능	최대 입력 토큰 수
Text-embedding-3-small	62,500	62.3%	8191
Text-embedding-3-large	9,615	64.6%	8191
Text_embedding-ada-002	12,500	61.0%	8191

MTEB는 Massive Text Embedding Benchmark의 줄임말로 텍스트 임베딩 모델의 성능을 평가하는 기준입니다. Text-embedding-3-small 모델은 비용 대비 성능이 매우 뛰어나고, text-embedding-3-large 모델은 더 높은 정확도를 제공합니다. 제가 테스트해 보니 한글 문서에선 text-embedding-3-large 모델과 Text-embedding-3-small 모델의 성능 차이가 있었습니다. 한글 문서를 많이 다루는 이 책에서는 text-embedding-3-large 모델을 사용합니다.

Do it! 실습 벡터 DB와 리트리버

결과 파일: sec02/rag_practice.ipynb

all_splits의 모든 청크들을 벡터로 변환하여 크로마 DB에 저장하겠습니다. 앞서 만든 all_splits에는 PDF를 청크로 쪼개 놓은 Document 객체들이 있습니다. 이 객체들을 벡터화하여 저장하겠습니다.

1. 크로마 DB를 만들고 데이터를 불러오겠습니다.

크로마 DB 생성하고 데이터 불러오기 rag_practice.ipynb (12)

```
from langchain_chroma import Chroma
import os

persist_directory = '../chroma_store'—①
```

```
if not os.path.exists(persist_directory):
    print("Creating new Chroma store")
    vectorstore = Chroma.from_documents(
        documents=all_splits,
        embedding=embedding,
        persist_directory=persist_directory
    )
else:
    print("Loading existing Chroma store")
    vectorstore = Chroma(
        persist_directory=persist_directory,
        embedding_function=embedding
    )
```

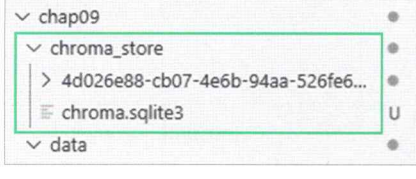
❶ 크로마DB를 생성하고 저장할 위치를 `persist_directory`로 지정합니다. 저장 경로는 '`../chroma_store`'로 설정했으며, 이 경로에 크로마DB가 생성되면 sqlite3 파일이 저장됩니다.

❷ 만약 해당 경로가 없다면 `os.path.exists()`로 이 디렉터리가 있는지 확인한 후 새로 생성합니다.

❸ 랭체인에서 크로마 DB를 쉽게 사용할 수 있도록 지원하고 있으므로 `langchain_chroma`의 `Chroma`를 이용해 벡터를 `vectorstore`에 저장합니다. 앞서 생성한 청크 `all_splits`를 `documents`에 넣고 설정한 임베딩 모델을 지정합니다. 마지막으로 `persist_directory`는 크로마 DB의 저장 위치를 의미합니다. `persist_directory`를 설정하지 않더라도 RAG를 사용할 수 있지만 이 경우 실제 DB가 생성되지 않고 데이터는 메모리에만 저장되므로 프로그램을 종료하면 해당 데이터는 사라집니다. `persist_directory`를 설정하면 요금을 내고 임베딩한 결과를 저장해 놓을 수 있습니다.

❹ 만약 `../chroma_store`가 이미 존재한다면 임베딩 과정을 반복할 필요 없이 기존 데이터를 바로 이용할 수 있도록 `else`에 관련 코드를 작성합니다. `Chroma`에 `persist_directory`와 `embedding_function`만 지정하면 됩니다.

셀을 실행하면 현재 폴더에 chroma_store라는 폴더가 생기고 그 안에 chroma.sqlite3 파일이 저장됩니다.



2. 다음과 같이 vectorstore에서 청크 3개를 가져오도록 설정한 retriever를 만들었습니다. `retriever.invoke()` 안에 텍스트를 입력하면 해당 내용이 벡터로 임베딩된 뒤, 크로마 DB에서 가장 유사한 청크 3개를 가져옵니다. 여기에서는 서울시의 환경 정책을 물어보았습니다.

유사 청크 가져와 질문하기 rag_practice.ipynb(13)

```
retriever = vectorstore.as_retriever(k=3)
docs = retriever.invoke("서울시의 환경 정책이 궁금해.")

for d in docs:
    print(d)
    print('------')
```

실행해 보면 300쪽 분량의 문서에서 서울시의 환경 관련 계획을 찾아낼 수 있습니다.

```
page_content='78 제3장 부문별 전략계획
제4절
기후·환경 부문
1. 개요
┌ 기후변화는 21세기에 전 지구적으로 가장 위중한 영향을 미칠 것으로 예상되며, 시민 생활의 모든 측
면과 연관되어 있어 향후 서울시의 적극적인 대응이 필요하다.
(... 생략 ...)
3-3 사람과 자연의 공존을 위한
친환경 생태도시 구축
3-3-1 건물 에너지 분야 효율성 개선 및 도심 속 생물 다양성 확보' metadata={'page': 86,
'source': 'C:/github/gpt_agent_2025_easyspub/data/2040_seoul_plan.pdf '}
------
page_content='180 제6장 계획의 실현
부문 지표명 지표의 통계명 내용 및 산정식 출처 목표치 및
관리방향
기후
·
환경
(... 생략 ...)
등)으로 인한 사고 및 사망자 수
행정안전부
재난연감 감소
감염병 제1급감염병 환자
수(비정기)' metadata={'page': 188, 'source': 'C:/github/gpt_agent_2025_easyspub/
data/2040_seoul_plan.pdf '}
------
(... 생략 ...)
```

Do it! 실습 주어진 청크에 기반하여 언어 모델로 답변 생성하기

결과 파일: sec02/rag_practice.ipynb

사용자가 질문하면 리트리버를 이용하여 관련 내용을 찾고 이를 바탕으로 언어 모델을 사용해 답변을 생성하는 기능을 개발하겠습니다.

1. 주어진 청크를 바탕으로 사용자의 질문에 답변을 생성하는 시스템을 설정합니다.

청크를 바탕으로 답변하는 시스템 생성하기 rag_practice.ipynb(14)

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder ①
from langchain.chains.combine_documents import create_stuff_documents_chain ②
from langchain_openai import ChatOpenAI
                                                                            ③
chat = ChatOpenAI(model="gpt-4o-mini")

question_answering_prompt = ChatPromptTemplate.from_messages(               ④
    [
        (
            "system",
            "사용자의 질문에 대해 아래 context에 기반하여 답변하라.:\n\n{context}",  ⑤
        ),
        MessagesPlaceholder(variable_name="messages"),                      ⑥
    ]
)

document_chain = create_stuff_documents_chain(chat, question_answering_prompt) ⑦
```

① ChatPromptTemplate은 언어 모델에 전달할 프롬프트를 정의하는 템플릿 클래스입니다. 이 템플릿을 사용해 시스템의 역할과 메시지를 설정합니다. MessagePlaceholder는 사용자 메시지와 같은 동적 데이터를 프롬프트에 삽입하기 위해 사용하는 클래스입니다. 이를 통해 여러 메시지 입력을 처리할 수 있습니다.

② create_stuff_documents_chain은 여러 텍스트를 결합해 답변 생성 체인을 만드는 함수입니다. 기존 프롬프트를 기반으로 언어 모델을 통해 최종 답변을 생성합니다.

③ 오픈AI의 GPT 모델을 사용하기 위해 ChatOpenAI를 임포트하고 chat이라는 객체를 생성합니다. 모델은 저렴한 gpt-4o-mini를 사용합니다.

④ 답변을 생성할 때 사용할 프롬프트 템플릿을 생성합니다.

⑤ 시스템 역할을 설정하고 주어진 컨텍스트를 바탕으로 답변하도록 모델에게 지시합니다. {context}는 문서 청크를 포함하는 자리 표시자로, 검색된 컨텍스트를 모델에 전달하는 빈칸 역할을 합니다.

⑥ messages 변수를 자리 표시자로 설정하여 사용자와 대화를 나눈 메시지를 동적으로 삽입할 수 있게 합니다. 이를 통해 사용자가 한 질문을 바탕으로 답변을 생성할 수 있습니다.

7 document_chain 객체는 검색된 문서 청크와 사용자 질문을 바탕으로 언어 모델을 호출하고 답변을 생성하는 역할을 합니다.

이제 질문을 할 준비가 거의 끝나 갑니다. 랭체인에서 제공하는 도구 가운데 ChatMessage History를 사용하면 채팅 메시지를 손쉽게 저장할 수 있습니다.

2. ChatMessageHistory의 객체인 chat_history를 만들고 add_user_message와 add_ai_message 메서드를 이용해 사용자와 AI의 메시지를 차례로 저장할 수 있습니다. 앞서 만든 document_chain을 사용하여 생성된 답변을 add_ai_message로 chat_history에 추가하고 그 후 결과를 출력하는 코드를 작성합니다.

메시지 저장하고 답변 출력하기 rag_practice.ipynb(15)

```
from langchain.memory import ChatMessageHistory

# 채팅 메시지를 저장할 메모리 객체 생성
chat_history = ChatMessageHistory()
# 사용자 질문을 메모리에 저장
chat_history.add_user_message("서울시의 온실가스 저감 정책에 대해 알려줘.")

# 문서 검색하고 답변 생성
answer = document_chain.invoke(
    {
        "messages": chat_history.messages,
        "context": docs,
    }
)

# 생성된 답변 메모리에 저장
chat_history.add_ai_message(answer)

print(answer)
```

실행해 보면 실제 PDF 문서를 참고하여 추출한 내용으로 그럴싸한 답변이 생성됩니다. 답변 내용은 서울시의 온실가스 저감 정책에 관한 구체적인 사항을 잘 설명하고 있습니다.

```
'서울시는 온실가스 저감을 위해 다양한 정책과 대책을 마련하고 있습니다. 주요 내용은 다음과 같습니다:

1. **건물 에너지 효율화**: 2026년까지 100만 호의 건물 에너지 효율화 사업을 추진하여, 기존 및 신규 건물의 제로에너지화를 촉진합니다. 이는 건물에서 발생하는 온실가스를 줄이기 위한 중요한 목표입니다.
```

```
2. **제로에너지 건물 의무화**: 신규 건물에 대해 제로에너지 건물(ZEB) 의무화를 통해 에너지 소비를 최소화하고, 신재생 에너지를 적극적으로 활용할 수 있도록 합니다.

3. **수송 부문 배출 감축**: 전기차 비중을 2026년까지 10%로 확대할 계획이며, 이를 위해 22만기의 전기차 충전 인프라를 구축할 예정입니다.
4. **기후예산제 도입**: 모든 예산사업에 온실가스 감축 방안을 고려하고 반영하기 위해 기후예산제를 시범 운영하고 있습니다.

5. **시민 참여 및 협력 체계 강화**: 시민과 정부, 기업 간의 협력을 통해 기후 행동을 활성화하고, 자원순환 문화를 확산시키는 다양한 캠페인과 교육 프로그램을 운영합니다.

이 외에도 서울시는 기후변화에 대응하기 위한 다양한 전략과 정책을 수립하여 시민의 생활과 안전을 보호하고, 도시의 회복력을 강화하는 데 집중하고 있습니다.'
```

3. 마지막으로 현재까지 chat_history에 저장된 대화 기록을 출력해 봅시다.

대화 기록 출력하기 — rag_practice.ipynb(16)

```
for m in chat_history.messages:
    print(m)
```

출력된 결과에는 사용자와 AI의 답변이 저장되어 있습니다.

```
content='서울시의 온실가스 저감 정책에 대해 알려줘.' additional_kwargs={} response_metadata={}
content=' 서울시는 온실가스 저감을 위해 다양한 정책과 대책을 마련하고 있습니다.
(... 생략 ...)
```

Do it! 실습 질의 확장 구현하기

결과 파일: sec02/rag_practice.ipynb

서울시 온실가스 저감 정책에 관한 대화에 이어서 사용자가 '뉴욕은?'라고 질문한다면 그 의도는 '뉴욕의 온실가스 저감 정책은 뭐야?'일 가능성이 큽니다. 이를 올바르게 처리하려면 질문을 문맥에 맞게 확장하여 관련된 청크를 검색하고 답변을 생성해야 합니다. 이 과정에서 질문을 '뉴욕은?'에서 '뉴욕의 온실가스 저감 정책은 뭐야?'와 같은 구체적인 형태로 변환하는 질의 확장이 필요합니다.

1. 필요한 라이브러리를 불러옵니다. StrOutputParser는 언어 모델이 생성한 응답을 문자열로 변환해 주는 역할을 합니다.

문자열 출력 파서 불러오기 — rag_practice.ipynb(17)

```
from langchain_core.output_parsers import StrOutputParser
```

2. query_for_nyc에 '뉴욕은?'이라는 질의 확장이 필요한 질문을 설정합니다. 이 질문을 기존의 대화 내용을 활용해 구체적인 질문으로 변환하겠습니다. ChatPromptTemplate.from_messages로 질문을 확장하는 프롬프트를 작성합니다. MessagesPlaceholder로 기존 대화 내용을 전달할 자리를 마련하고 시스템 메시지로 의도를 파악한 후 질문을 명확하게 변환하도록 지시합니다.

구체적인 질문으로 바꾸기 — rag_practice.ipynb(18)

```
query_for_nyc = "뉴욕은?"

# query augmentation
# 기존 대화 내용을 활용해 query_augmentation 수행
query_augmentation_prompt = ChatPromptTemplate.from_messages(
    [
        MessagesPlaceholder(variable_name="messages"), # 기존 대화 내용
        (
            "system",
            "기존의 대화 내용을 활용하여 사용자가 질문한 의도를 파악해서 한 문장의 명료한 질문으로 변환하라. 대명사나 이, 저, 그와 같은 표현을 명확한 명사로 표현하라. :\n\n{query}",
        ),
    ]
)
```

3. 앞에서 ChatOpenAI로 정의한 chat 객체와 프롬프트를 연결하고 결과를 문자열로 변환하기 위해 StrOutputParser를 사용합니다.

결과 문자열로 변환하는 체인 만들기 — rag_practice.ipynb(19)

```
query_augmentation_chain = query_augmentation_prompt | chat | StrOutputParser()
```

4. 이렇게 만든 query_augmentation_chain을 실행하려면 사용자의 질문과 이전 대화 내용을 함께 제공해야 합니다. messages와 query 변수를 딕셔너리 형식으로 묶어 invoke 메서드를 사용해 전달하면 질문이 더 명확하게 변환됩니다.

질문을 더 명확하게 변환하기 rag_practice.ipynb(20)

```
augmented_query = query_augmentation_chain.invoke({
    "messages": chat_history.messages,
    "query": query_for_nyc
})

print(augmented_query)
```

실행해 보면 언어 모델이 '뉴욕은?'이라는 질문을 '뉴욕의 온실가스 저감 정책에 대해 알려주세요'라는 명확한 질문으로 변환합니다. 이렇게 구체적이고 명료한 질문으로 변환해야 리트리버를 이용해 이 질문과 관련된 청크들을 가져와 적절한 답변을 생성할 수 있습니다.

```
뉴욕시의 온실가스 저감 정책에 대해 알려주세요
```

5. 이렇게 만들어진 augmented_query를 이용해 관련된 내용을 검색할 수 있습니다. 이전에 썼던 코드를 활용해 리트리버를 실행하고 결과를 출력했습니다.

리트리버 실행하고 결과 출력하기 rag_practice.ipynb(21)

```
docs = retriever.invoke(augmented_query)

for d in docs:
    print(d)
    print('------')
```

셀을 실행해 보면 「One NYC 2050」 문서에서 온실가스와 대기오염에 관한 내용이 검색되었습니다.

```
page_content='Particulate matter (PM2.5) is estimated to contribute to more
than 2,000 deaths and just under 6,000 emergency room visits
and hospitalizations for cardiovascular and respiratory disease
each year.
```

```
(... 생략 ...)
Over the next four years, New York City will expand
initiatives to curtail vehicular emissions, working with the' metadata={'page': 186,
'source': 'C:/github/gpt_agent_2024_book/chap10/data/OneNYC_2050_Strategic_Plan.pdf'}
------
page_content='NYC.GOV/O neNYC14    ¦    O neNYC 2050 : A LIVABLE CLIMATEWhile we are
working in coordination with our state, regional,
and market
(... 생략 ...)
We envision a 50 percent renewable electricity grid by 2030.' metadata={'page': 241,
'source': 'C:/github/gpt_agent_2024_book/chap10/data/OneNYC_2050_Strategic_Plan.pdf'}
------
(... 생략 ...)
```

6. 이 내용을 활용해 언어 모델로 답변을 생성할 수 있습니다. 앞에서 작성했던 코드를 그대로 활용하되 .add_user_message에 '뉴욕은?'이라는 질문이 담긴 query_for_nyc를 추가합니다.

언어 모델에서 답변 생성하기 — rag_practice.ipynb(22)

```
chat_history.add_user_message(query_for_nyc)  # query_for_nyc에 "뉴욕은?" 추가

answer = document_chain.invoke(
    {
        "messages": chat_history.messages,
        "context": docs,
    }
)

# 생성된 답변 메모리에 저장
chat_history.add_ai_message(answer)

print(answer)
```

실행해 보면 뉴욕의 정책에 관한 내용이 잘 출력됩니다.

```
뉴욕시는 온실가스 저감을 위해 여러 가지 정책과 이니셔티브를 추진하고 있습니다. 주요 내용은 다음과 같
습니다:
1. **청정 전력 자원 확대**: 뉴욕시는 2040년까지 100% 청정 전력 자원을 확보하는 목표를 세우고 있으
며, 2030년까지는 50%의 재생 가능 전력 그리드를 구현할 계획입니다.
(... 생략 ...)
이러한 노력들을 통해 뉴욕시는 기후 변화에 대응하고, 지속 가능한 도시로 발전하기 위해 힘쓰고 있습니다.
```

09-3 스트림릿으로 챗봇 완성하기

08-5절에서 만들었던 스트림릿 코드를 응용해서 RAG을 활용한 챗봇을 완성해 보겠습니다.

Do it! 실습 기본 스트림릿 코드에 리트리버 추가하기

결과 파일: sec03/retriever.py, rag_1.py

1. 09-2절에서 만든 코드를 활용해 질의응답 시스템을 만들겠습니다. retriever.py 파일을 만들고 rag_practice.ipynb 파일에서 관련 코드를 복사해서 가져옵니다. 그리고 `persist_directory`의 폴더 경로를 절대 경로로 수정합니다. 절대 경로로 설정하면 어떤 폴더에서 실행하더라도 크로마 DB의 위치를 확실하게 지정할 수 있습니다. 크로마 DB는 앞의 예제에서 만들었던 DB를 그대로 사용하겠습니다.

크로마 DB와 임베딩 모델을 활용한 질의응답 시스템 만들기 — retriever.py

```
# 임베딩 모델 선언하기
from langchain_openai import OpenAIEmbeddings
embedding = OpenAIEmbeddings(model='text-embedding-3-large')

# 언어 모델 불러오기
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o")

# Load Chroma store
from langchain_chroma import Chroma
print("Loading existing Chroma store")
persist_directory = 'C:/github/gpt_agent_2025_easyspub/chap09/chroma_store'  # 절대 경로

vectorstore = Chroma(
    persist_directory=persist_directory,
    embedding_function=embedding
)

# Create retriever
retriever = vectorstore.as_retriever(k=3)

# Create document chain
from langchain.chains.combine_documents import create_stuff_documents_chain
```

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.output_parsers import StrOutputParser  # 문자열 출력 파서 불러오기

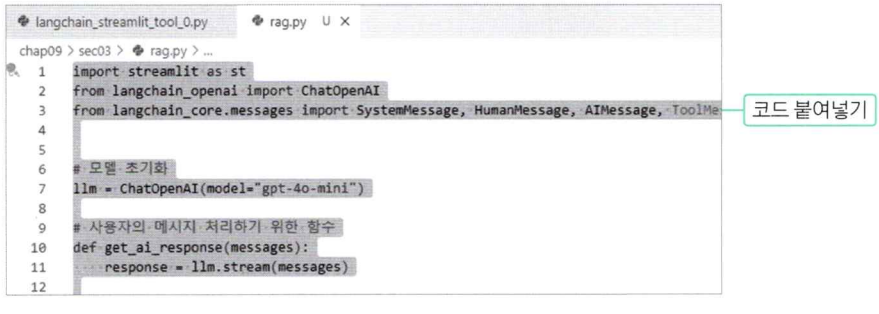
question_answering_prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "사용자의 질문에 대해 아래 context에 기반하여 답변하라.:\n\n{context}",
        ),
        MessagesPlaceholder(variable_name="messages"),
    ]
)

document_chain = create_stuff_documents_chain(llm, question_answering_prompt) |
StrOutputParser()

# query augmentation chain
query_augmentation_prompt = ChatPromptTemplate.from_messages(
    [
        MessagesPlaceholder(variable_name="messages"),  # 기존 대화 내용
        (
            "system",
            "기존의 대화 내용을 활용하여 사용자가 질문한 의도를 파악해서 한 문장의 명료한 질문으로
변환하라. 대명사나 이, 저, 그와 같은 표현을 명확한 명사로 표현하라. :\n\n{query}",
        ),
    ]
)

query_augmentation_chain = query_augmentation_prompt | llm | StrOutputParser()
```

2. RAG를 구현하기 전에 오픈AI의 API를 활용해 기본적인 대화를 할 수 있는 코드를 작성합니다. 08-5절 '랭체인 메모리 없이 멀티턴 만들기' 실습에서 사용한 코드를 활용하겠습니다. rag.py 파일을 새로 만들고 저장해 둔 langchain_streamlit_tool_0.py 파일의 코드를 붙여 넣으세요.



3. 복사한 코드에 질의 확장 기능을 추가하기 위해 다음과 같이 수정합니다.

앞에서 작성한 스트림릿 코드에 질의 확장 추가하기 rag.py

```
import streamlit as st
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage, ToolMessage

import retriever ①

# 모델 초기화
llm = ChatOpenAI(model="gpt-4o-mini")

# 사용자의 메시지를 처리하는 함수
def get_ai_response(messages):
    response = llm.stream(messages)

    for chunk in response:
        yield chunk

# Streamlit 앱
st.title("┌ GPT-4o Langchain Chat")

# 스트림릿 session_state에 메시지 저장
if "messages" not in st.session_state:
    st.session_state["messages"] = [
        SystemMessage("너는 문서에 기반해 답변하는 도시 정책 전문가야.") ②
    ]

# 스트림릿 화면에 메시지 출력
for msg in st.session_state.messages:
    if msg.content:
        if isinstance(msg, SystemMessage):
            st.chat_message("system").write(msg.content)
        elif isinstance(msg, AIMessage):
            st.chat_message("assistant").write(msg.content)
        elif isinstance(msg, HumanMessage):
            st.chat_message("user").write(msg.content)

# 사용자 입력 처리
if prompt := st.chat_input():
    st.chat_message("user").write(prompt)                    # 사용자 메시지 출력
    st.session_state.messages.append(HumanMessage(prompt))   # 사용자 메시지 저장
```

```
    print("user\t:", prompt)                                          ③

    augmented_query = retriever.query_augmentation_chain.invoke({
        "messages": st.session_state["messages"],
        "query": prompt,
    })
    print("augmented_query\t", augmented_query)

    with st.spinner(f"AI가 답변을 준비 중입니다... '{augmented_query}'"):   ⑤
        response = get_ai_response(st.session_state["messages"])
        result = st.chat_message("assistant").write_stream(response)  # AI 메시지 출력
    st.session_state["messages"].append(AIMessage(result))            # AI 메시지 저장
```

④

4칸 들여쓰기

① 앞에서 만든 retriever 파일을 임포트합니다.

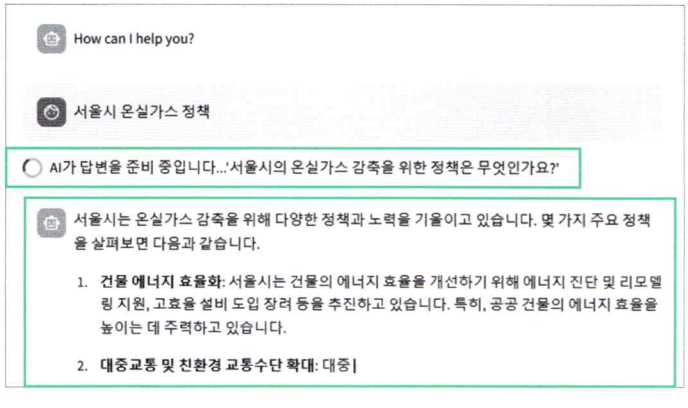
② 이 크로마 DB에는 서울과 뉴욕의 도시 정책 계획 문서가 담겨 있으므로 프롬프트도 이에 맞게 '문서에 기반해 답변하는 도시 정책 전문가'라고 수정합니다.

③ 사용자가 입력한 내용(prompt)을 터미널 창에 출력합니다.

④ 이제 사용자가 입력한 prompt를 이용해 예전에 했던 것처럼 확장된 질의를 만듭니다. messages와 query 변수를 딕셔너리 형식으로 묶어 invoke 메서드를 사용해 전달하면 질문이 더 명확하게 변환됩니다. 이 코드는 09-2절 '질의 확장 구현하기' 실습에서도 다뤘습니다.

⑤ 어떤 질문에 답변을 생성하는지를 보여 주는 요소와 문구를 추가합니다.

터미널 창에 `streamlit run` *파일명*`.py`을 입력해 코드를 실행하면 다음처럼 'AI가 답변을 준비 중입니다...' 라는 문구가 나오고 나서 확장된 질의가 출력됩니다. 하지만 아직 문서와 관계없는 일반적인 답변만 생성합니다.



4. RAG를 구현하기 위해 리트리버를 추가합니다. rag.py 파일의 get_ai_response 함수를 수정하면 됩니다. 기존에는 매개변수가 messages뿐이었지만 docs를 추가하여 관련 문서를 같이 받아 문서에 기반한 답변을 생성하도록 수정합니다.

검색 문서에 기반한 답변 생성하기 — rag.py

```
(... 생략 ...)

# 모델 초기화
llm = ChatOpenAI(model="gpt-4o-mini")

# 사용자의 메시지를 처리하는 함수
def get_ai_response(messages, docs):
    response = retriever.document_chain.stream({
        "messages": messages,
        "context": docs
    })

    for chunk in response:
        yield chunk
(... 생략 ...)
```

5. get_ai_response 함수를 사용하기 위해 코드를 추가합니다. retriever.py에 있는 retriever를 이용해 관련 문서를 검색해서 docs에 담고 그 결과를 출력합니다. 그리고 get_ai_response 함수에 docs를 추가해서 실행합니다

리트리버 및 답변 생성 추가하기 — rag.py

```
(... 생략 ...)

augmented_query = retriever.query_augmentation_chain.invoke({
        "messages": st.session_state["messages"],
        "query": prompt,
    })
    print("augmented_query\t", augmented_query)

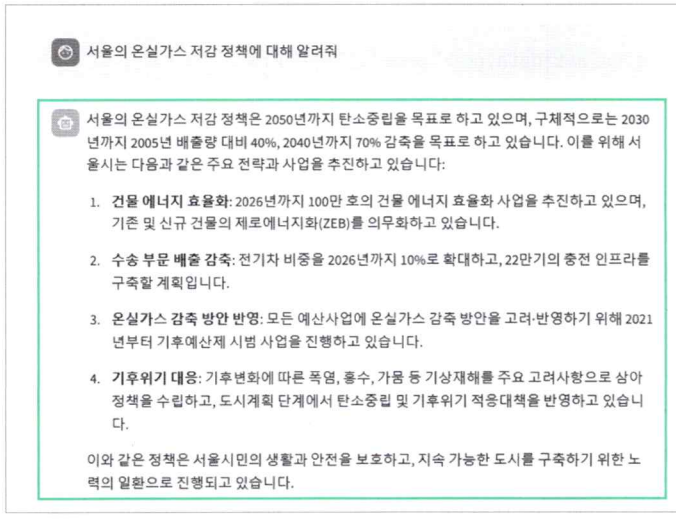
    # 관련 문서 검색
    print("관련 문서 검색")
    docs = retriever.retriever.invoke(f"{prompt}\n{augmented_query}")

    for doc in docs:
        print('---------------')
```

```
        print(doc)
    print("===============")

    with st.spinner(f"AI가 답변을 준비 중입니다... '{augmented_query}'"):
        response = get_ai_response(st.session_state["messages"], docs)
        result = st.chat_message("assistant").write_stream(response)  # AI 메시지 출력
    st.session_state["messages"].append(AIMessage(result))            # AI 메시지 저장
```

이 코드를 저장한 후 스트림릿을 실행하면 문서에서 리트리버로 검색한 내용을 바탕으로 구체적인 답변을 해줍니다.



Do it! 실습 출처 표기하기

결과 파일: sec03/rag.py

RAG를 사용할 때 언어 모델의 답변이 실제 문서에 근거한 것인지 아니면 엉뚱한 문서를 참고했거나 모델이 임의로 생성한 답변인지 확인해야 할 때가 있습니다. 답변에 사용한 문서의 출처가 스트림릿에 표기되도록 구현해 보겠습니다.

기존에는 `get_docs` 함수로 가져온 청크들을 `print`로 터미널 창에 출력하게 되어 있었는데, 이를 스트림릿에서도 표시할 수 있도록 수정합니다.

리트리버로 검색한 문서 출처 표기하기 rag.py

```
(... 생략 ...)
    # 관련 문서 검색
    print("관련 문서 검색")
    docs = retriever.retriever.invoke(f"{prompt}\n{augmented_query}")

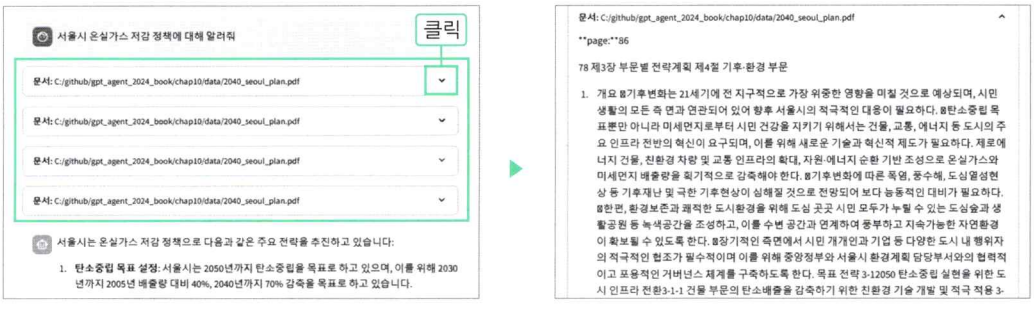
    # 검색된 문서의 출처 표시
    for doc in docs:
        print('---------------')
        print(doc)

        with st.expander(f"**문서:** {doc.metadata.get('source', '알 수 없음')}"):
            # 파일명과 페이지 정보 표시
            st.write(f"**page:**{doc.metadata.get('page', '')}")
            st.write(doc.page_content)
    print("===============")

    with st.spinner(f"AI가 답변을 준비 중입니다... '{augmented_query}'"):
        response = get_ai_response(st.session_state["messages"], docs)
        result = st.chat_message("assistant").write_stream(response)  # AI 메시지 출력
    st.session_state["messages"].append(AIMessage(result))            # AI 메시지 저장
```

`streamlit run` *파일명*`.py`로 코드를 실행해 봅시다. 챗봇에 질문하면 답변이 생성되기 전에 사용한 청크들을 드롭다운 형태로 보여 줍니다. 클릭하면 해당 청크의 내용을 확인할 수 있습니다.

✦ 드롭다운 형태란 사용자가 클릭하면 숨어 있는 내용을 표시해서 확인할 수 있는 인터페이스를 말합니다.



RAG에 기반해 챗봇을 만들 때에는 사용자에게 고품질 답변을 제공할 수 있도록 최선을 다해야 합니다. 그러나 문서의 특성이나 여러 이유로 사용자가 RAG를 통한 답변의 품질이 만족스럽지 않을 수 있으므로 과도하게 신뢰하면 문제가 발생할 수 있습니다. 따라서 언어 모델이 어떤 문서를 기반으로 답변을 생성했는지 사용자가 확인할 수 있도록 장치를 마련해야 사용자와 개발자 모두 보호할 수 있습니다.